Entity Relationship Model

Unit - 2

Entities and Attributes

- **Entity:**
  - A thing or an object.
  - It has set of properties
  - Values for some set of properties, it can uniquely identify it
  - Entities are specific objects or things in the mini-world that are represented in the database.
  - For example the EMPLOYEE John Smith, the Research DEPARTMENT, the ProductX PROJECT

- Attributes are properties used to describe an entity.
  - For example an EMPLOYEE entity may have the attributes Name, SSN, Address, Sex, BirthDate
- A specific entity will have a value for each of its attributes.
  - For example a specific employee entity may have Name='John Smith', SSN='123456789', Address ='731, Fondren, Houston, TX', Sex='M', BirthDate='09-JAN-55‘

- Each attribute has a value set (or data type) associated with it – e.g. integer, string, subrange, enumerated type, …

Type of Attributes

- Simple Attribute
  - Each entity has a single atomic value for the attribute.
  - For example: SSN or Gender.
- Composite Attribute
  - An attribute where the values of that attribute can be further subdivided into meaningful sub-parts."
  - For Example: Emp_Name – may be stored as first name, last name, middle initial
  - Address(Apt#, House#, Street, City, State, ZipCode, Country)
- Multivalue Attribute
  - An attribute that can have more than one value associated with the key of the entity.
  - For Example: Degree of a person, Hobby of a person, Bank branch Location, Color of a car etc.
- Derived Attribute
  - An attribute whose value is calculated (derived) from other attributes.
  - The derived attribute need not be physically stored within the database; instead, it can be derived by using an algorithm.
  - For Example: Age field (can be derived from Birth_date field)

Cont…

- In general, composite and multi-valued attributes may be nested arbitrarily to any number of levels, although this is rare.
  - For example, PreviousDegrees of a STUDENT is a composite multi-valued attribute denoted by {PreviousDegrees (College, Year, Degree, Field)}
- Multiple PreviousDegrees values can exist
- Each has four subcomponent attributes:
  - College, Year, Degree, Field

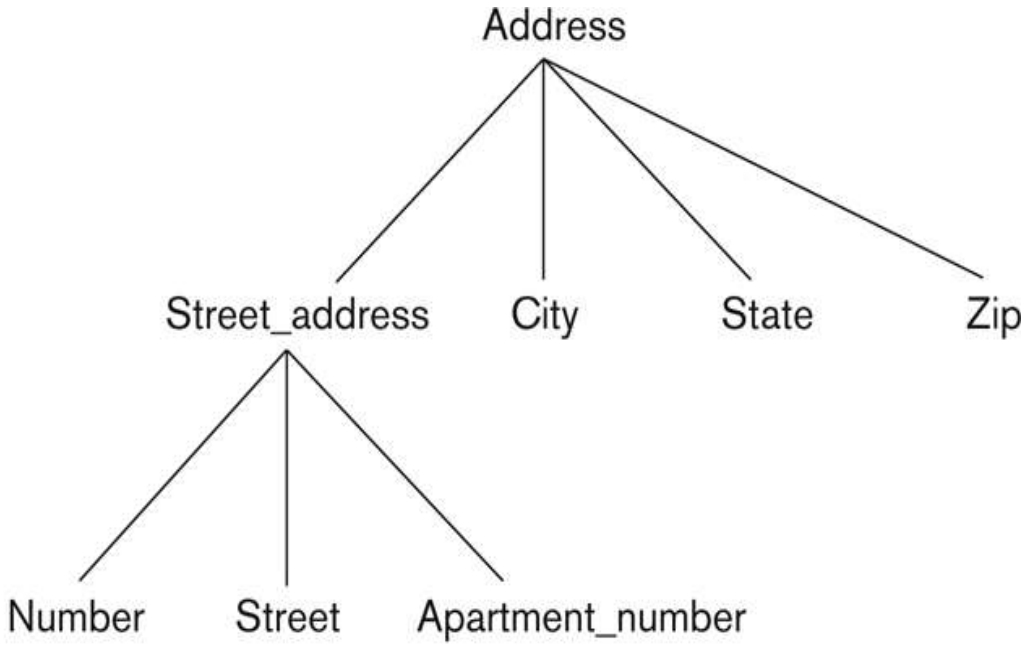


Figure 3.4
A hierarchy of composite attributes.

Entity Type and Entity Sets

- A database usually contains groups of entities that are similar
  - For Ex. Company Employees
- Each entity shares the same attribute but each entity has its own values for each attribute
- Entity Type: A collection (set of) of entities that have the same attributes
- A collection of all entities of a particular entity type in the database at any point in time is called an entity set or entity collection
- Entity set is usually referred by using the same name as the entity type.
  - For Ex.: EMPLOYEE refers to both type of entity as well as the collection ofall *employee* entities in the Database.

Relationship

- A relationship relates two or more distinct entities with a specific meaning.
  - Eg. Employee WORKS FOR Department -------”Work For” is the relationship
  - Student ENROLLS Course ----------”Enrolls” is the relationship
- A relationship type R among n entity types E1, E2, ..., En defines a set of associations—or a relationship set—among entities from these entity types.
- As for the case of entity types and entity sets, a relationship type and its corresponding relationship set are customarily referred to by the same name, R.

Relationship instances

- In the mini-world represented by figure below employees e1, e2, e3 and e6 work for department d1, e2 and e4 works for d2 and e5 and e7 work for d3.

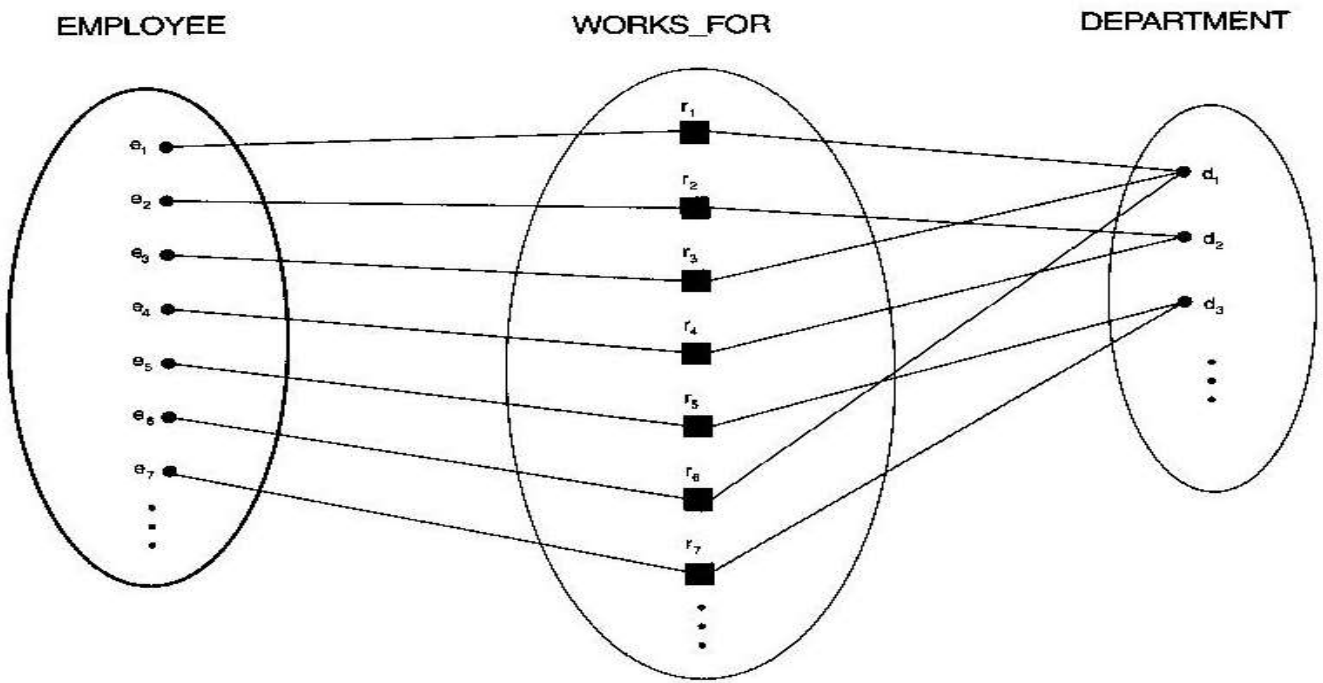
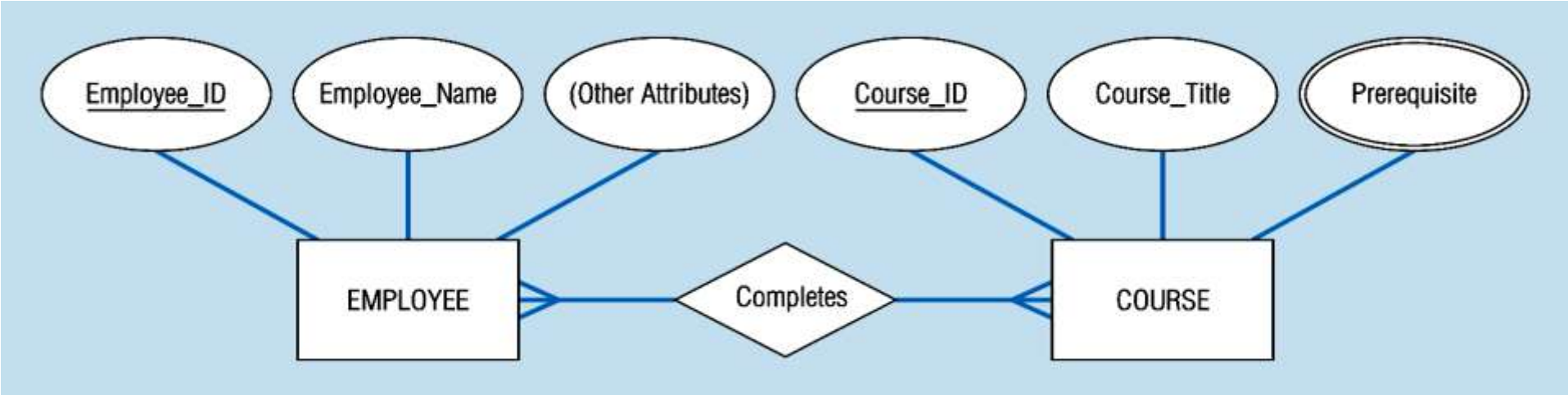
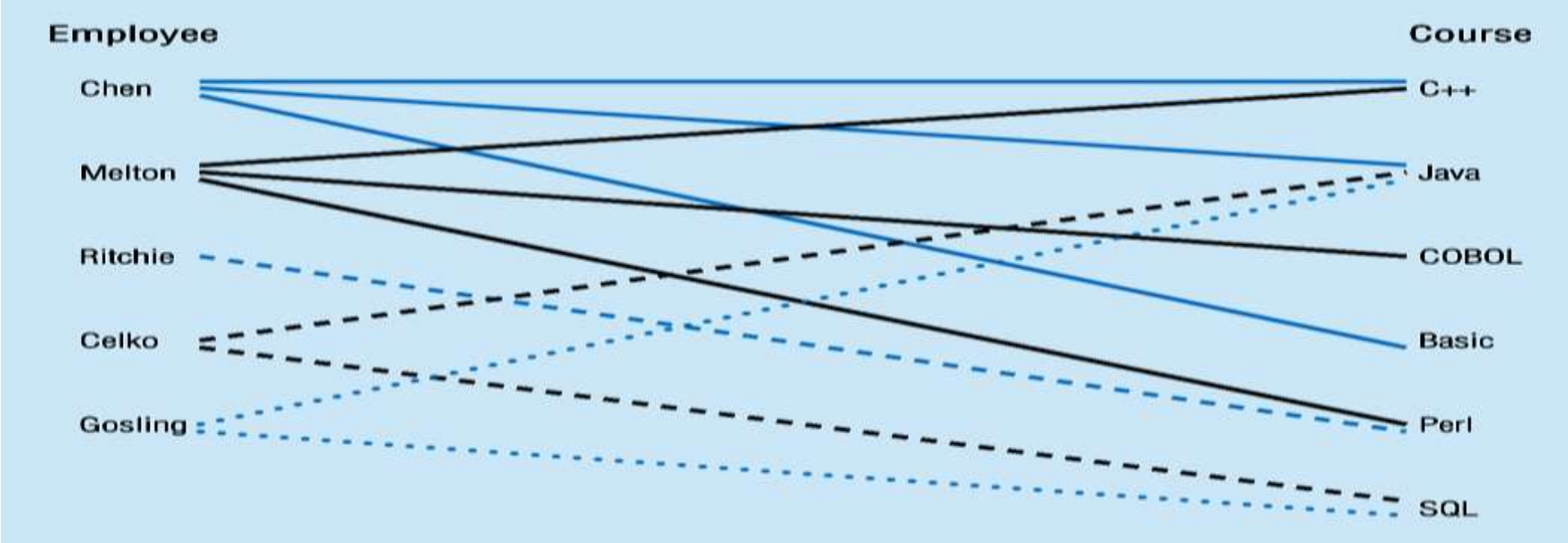


Figure 3.9 Some instances of the WORKS_FOR relationship between EMPLOYEE and DEPARTMENT.

Relationship types and instances





Relationship type vs. relationship set

- Previous figures displayed the relationship sets
- Each instance in the set relates individual participating entities – one from each participating entity type
- In ER diagrams, we represent the *relationship type* as follows:
  - Diamond-shaped box is used to display a relationship type
  - Connected to the participating entity types via straight lines

Relationship type vs. relationship set

- Relationship Type:
  - Is the schema description of a relationship
  - Identifies the relationship name and the participating entity types
  - Also identifies certain relationship constraints
- Relationship Set:
  - The current set of relationship instances represented in the database
  - The current *state* of a relationship type

Unit-2

Cont…

Cardinality and Participation Constraints

- In general, cardinality tells you "How many".
- Cardinality can be:
  - One to one (1:1): every time one entity occurs, there is exactly one occurrence of another entity.
  - One to many (1:m): every time one entity occurs, there are multiple occurrences of another entity.
  - Many to many (m:m): for each occurrence of an entity, there can be one or many occurrences of another—an vice versa.
- Participation Constraints:
  - Total Participation
  - Partial Participation

Example for 1:1 Relationship Type

- In an **Entity-Relationship (ER) Model**, a **1:1 (one-to-one) relationship** means:
- **One instance** of Entity A is associated with **only one instance** of Entity B, and vice versa.

◆ **Example 1: Person – Passport**

Person → Each person has **one passport**.

Passport → Each passport belongs to **one person** only.

◆ **Example 2: Husband – Wife**

Husband → one wife only.

Wife → one husband only.

📌 **Key Point**: In ER diagrams, the **1:1 relationship** is shown by connecting two entities with a line and placing **1** on both ends.

Example for 1:M Relationship Type

- In an **Entity-Relationship (ER) Model**, a **1:M (one-to-many) relationship** means:
- **One instance** of Entity A can be associated with **many instances** of Entity B.
- But each instance of Entity B is related to **only one instance** of Entity A.

◆ **Example 1: Teacher – Students**

Teacher → One teacher can teach **many students**.

Student → Each student is taught by **one teacher** (in this relationship).

◆ **Example 2: Department – Employees**

Department → One department can have **many employees**.

Employee → Each employee belongs to **only one department**.

◆ **Example 3: Mother – Children**

Mother → One mother can have **many children**.

Child → Each child has **one biological mother**

📌 **Key point:** "1" is placed on the side where only one instance exists, and "M" (many) is placed on the side where multiple instances exist.

Example for N:M Relationship Type

- In an **Entity-Relationship (ER) Model**, a **many-to-many (M:N) relationship** means:
- One instance of **Entity A** can be associated with **many instances** of **Entity B**,
- and one instance of **Entity B** can also be associated with **many instances** of **Entity A**.

◆ **Example 1: Students – Courses**

Student → One student can enroll in **many courses**.

Course → One course can have **many students** enrolled.

◆ **Example 2: Authors – Books**

Author → One author can write **many books**.

Book → One book can be written by **many authors**

◆ **Example 3: Doctors – Patients**

Doctor → A doctor can treat **many patients**.

Patient → A patient can be treated by **many doctors**.

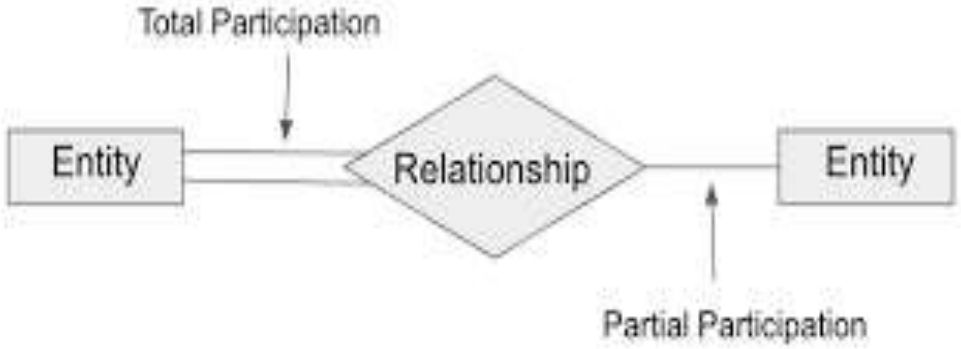
📌 **Key point:** In practice, an **M:N relationship is usually broken into two 1:M relationships** using a **bridge/associative entity** (like *Enrollment* between Student & Course).

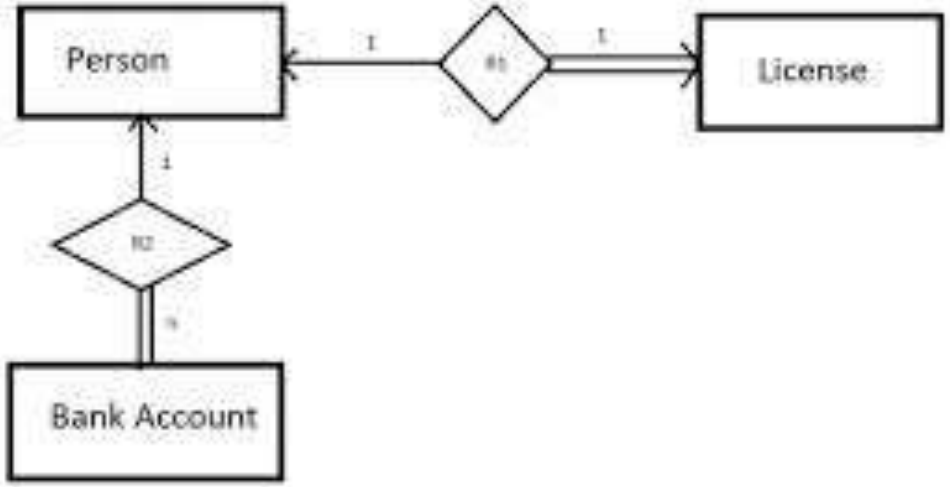
Participation Constraint

Example

- **Total Participation** – each student must enroll in a course
- **Partial Participation** – If some courses are not enrolled by any students, the participation of the course will be partial.

Participation in Enrolled relationship set: **Partial** Course
Total Student





Degree of a Relationship

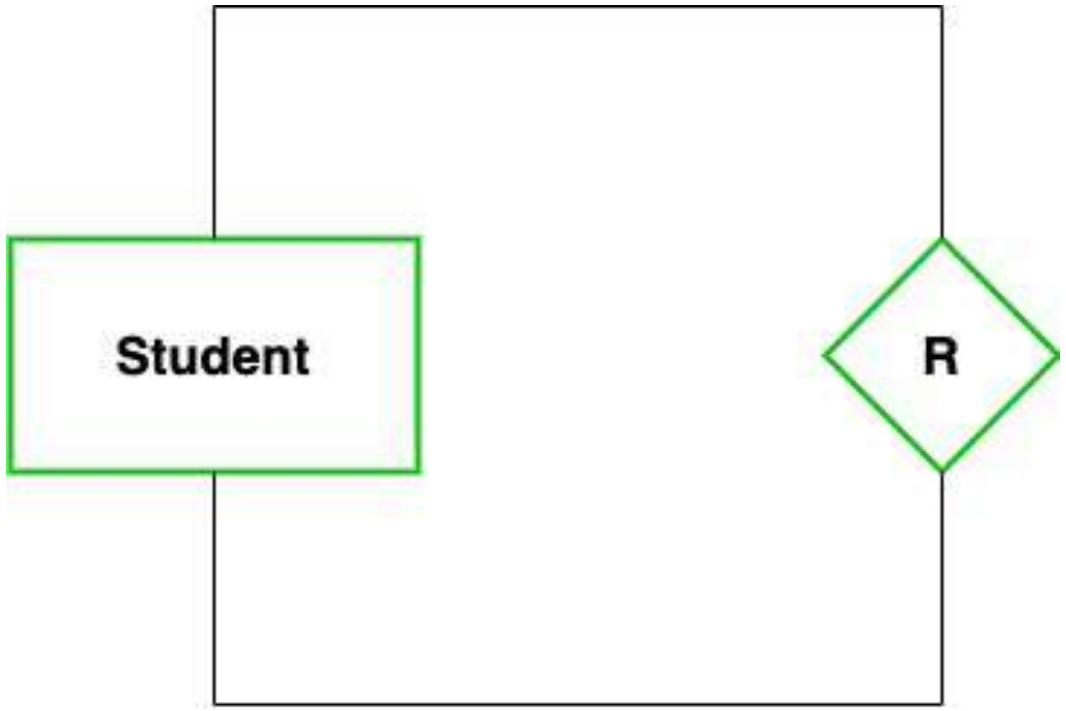
- Based on the number of linked entity types, we have 4 types of degrees of relationships.
  - Unary
  - Binary
  - Ternary
  - N-ary

Unary Relationship

- In this type of relationship, both the associating entity type are the same. So, we can say that unary relationships exist when both entity types are the same and we call them the degree of relationship is 1. Or in other words, in a relation only one entity set is participating then such type of relationship is known as a unary relationship.

- Example: In a particular class, we have many students, there are monitors too. So, here class monitors are also students. Thus, we can say that only students are participating here. So the degree of such type of relationship is 1.

Example – Unary Relationship

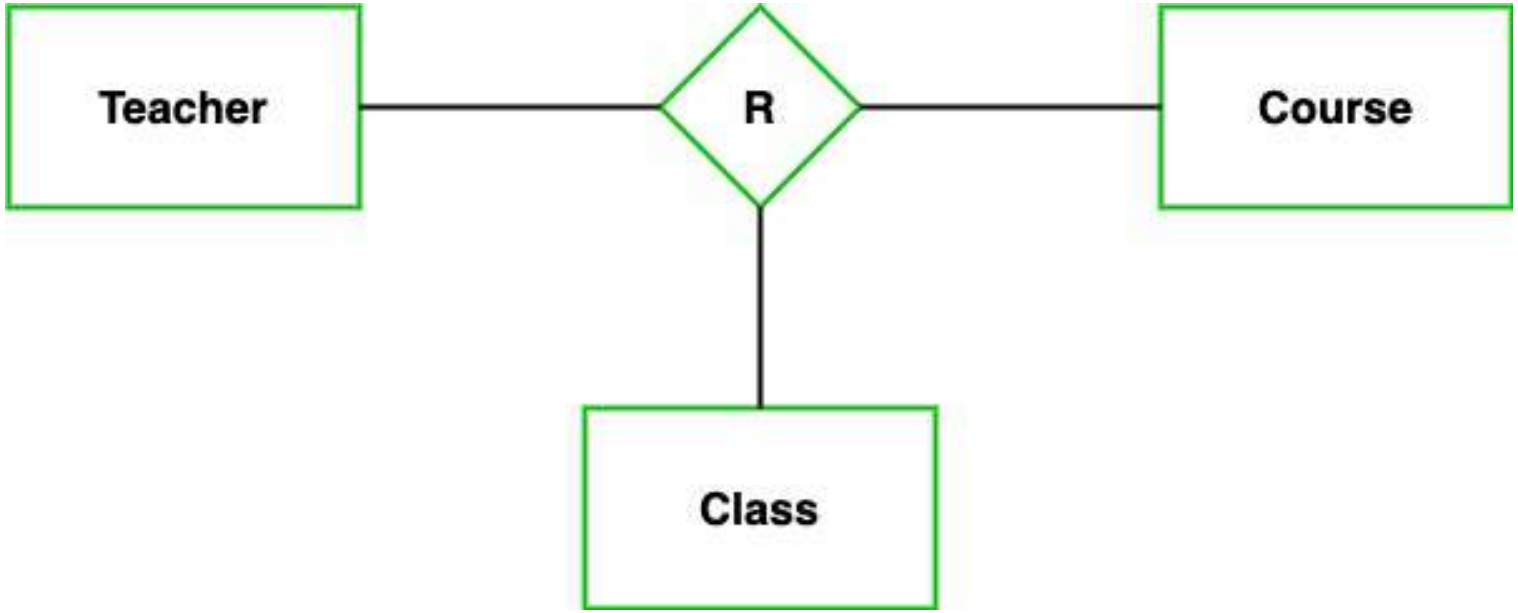


Binary Relationship

- In a Binary relationship, there are two types of entity associates.
- Binary relationship exists when there are two types of entity and we call them a degree of relationship is 2.
- Or in other words, in a relation when two entity sets are participating then such type of relationship is known as a binary relationship. This is the most used relationship and one can easily be converted into a relational table.

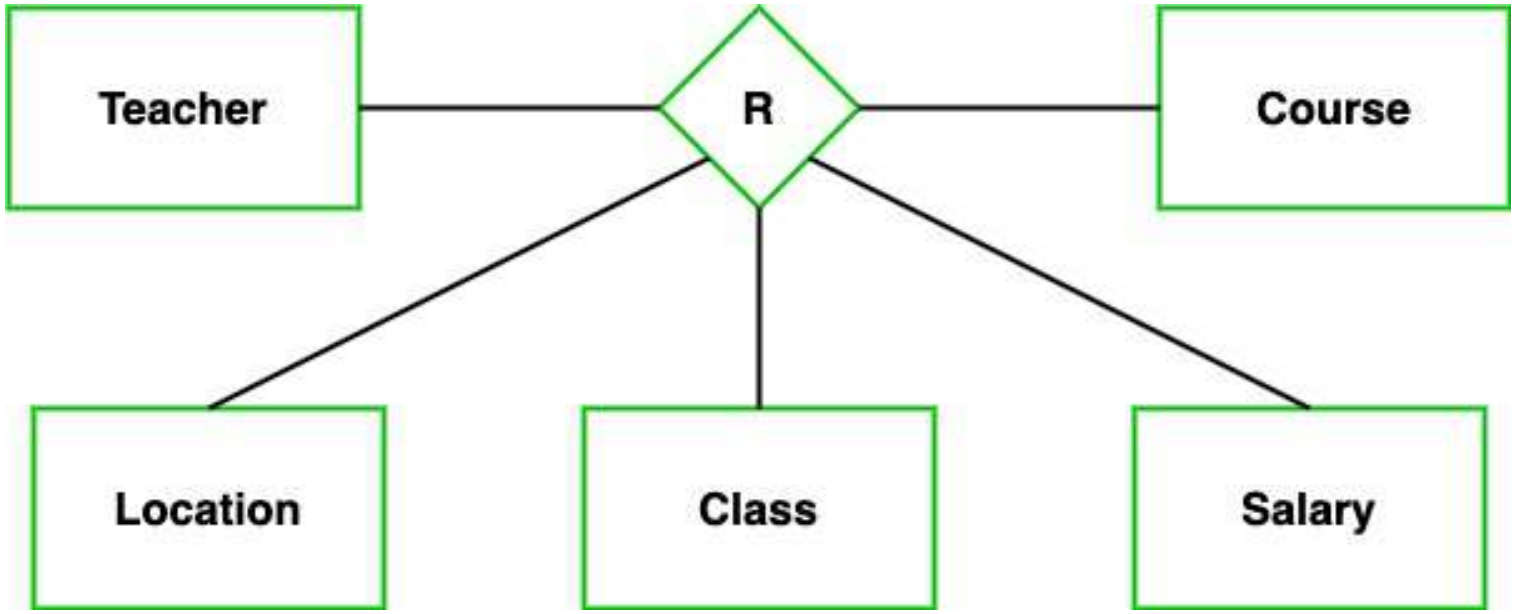
Ternary Relationship

- In the Ternary relationship, there are three types of entity associates.
- Ternary relationship exists when there are three types of entity and we call them a degree of relationship is 3



N-ary Relationship

- In the N-ary relationship, there are n types of entity that associates.
- N-ary relationship exists when there are n types of entities.



ER-Diagram

Unit-2 Cont…

Weak Entity Types

- An entity that does not have a key attribute
- A weak entity must participate in an identifying relationship type with an owner or identifying entity type
- Entities are identified by the combination of:
  - A partial key of the weak entity type
  - The particular entity they are related to in the identifying entity type
- **Example: 1**
  - A DEPENDENT entity is identified by the dependent's first name, *and* the specific EMPLOYEE with whom the dependent is related
  - Name of DEPENDENT is the *partial key*
  - DEPENDENT is a *weak entity type*
  - EMPLOYEE is its identifying entity type via the identifying relationship type DEPENDENT_OF

- **Example: 2**
  - ORDER (Strong Entity) and ORDERITEM (Weak Entity).
  - An OrderItem cannot exist without an Order.
  - Partial key: ItemNumber within an order.
  - Weak relationship: "Contains" between Order and OrderItem.

Company Management System

- Organization has multiple departments
- Employees work for the department and manages the department
- Per department there is an supervisor (an employee)
- Department executes different projects
- Employees work on those projects on hourly basis
- Organization manages the information of the Employee’s dependents also.

Example

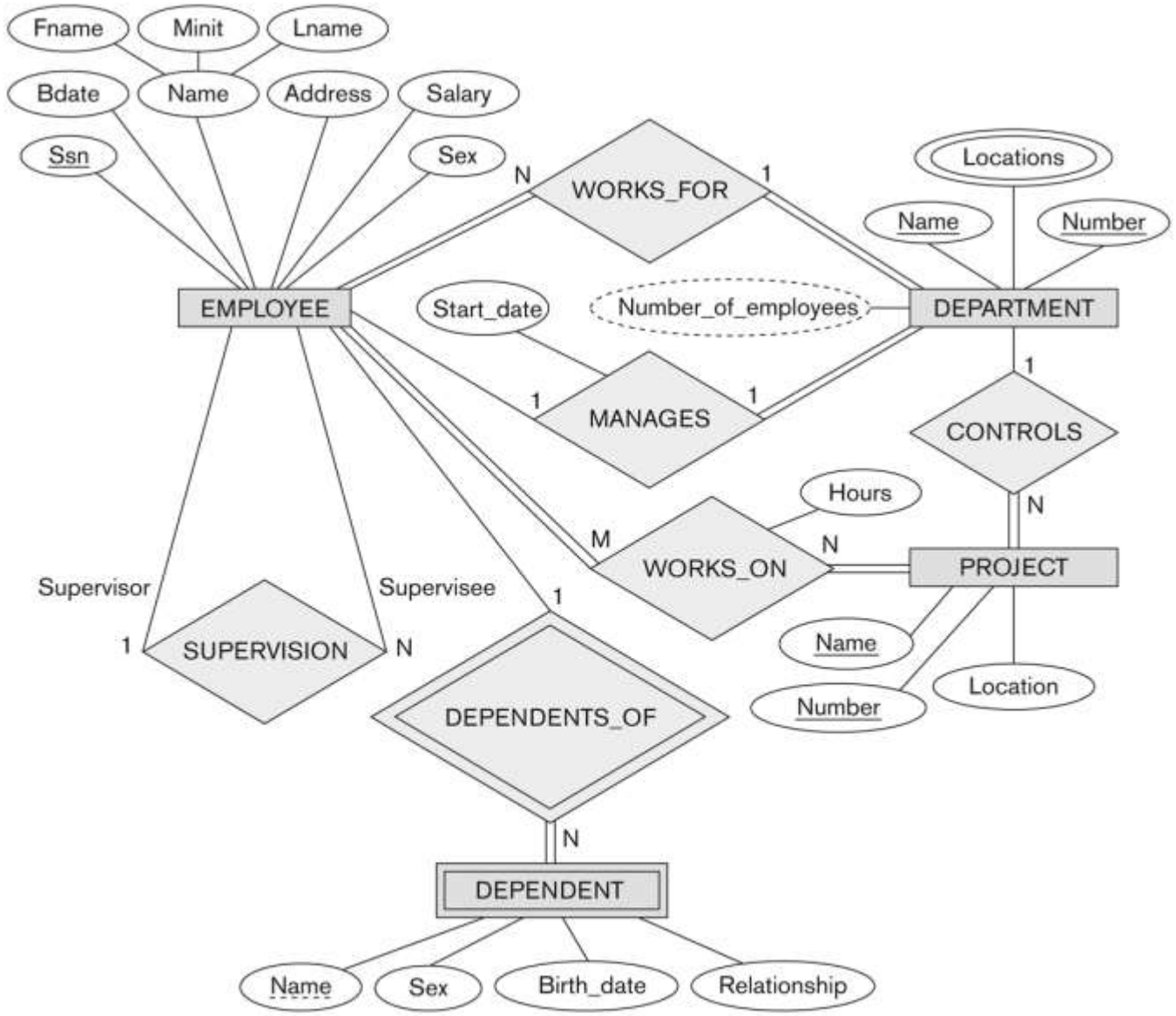


Figure 3.2
An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

University Management System

- Different College / institute are there under the university
- Each College has different departments that manages information related to courses
- Students are enrolled in a course
- Courses offers different subjects
- Faculties teaches subjects in different courses
- One faculty in each department is the head of that department
- One faculty is the director of the college / institute.
- Course has exam, students appear for the exam and get grades for the same

Draw an ER Diagram for the Library Management System.

- In a Library different types of books are purchased for the students to refer.
- Books are categorized by different authors and publishers.
- Student registers in the library and can issue books.
- Librarian manages student details.

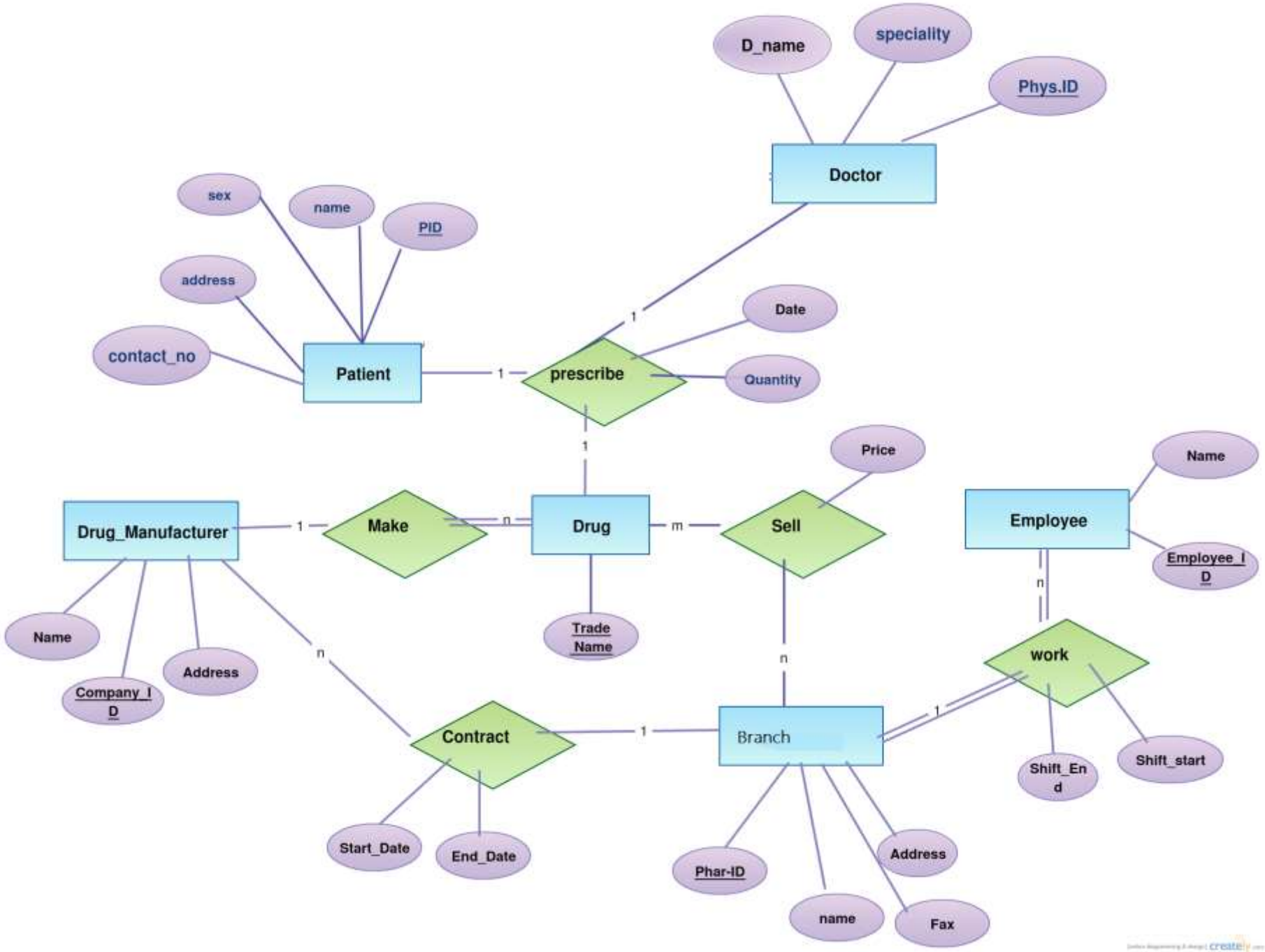
Project Management System of an IT Company

- A company handles different projects from clients
- Manages client information
- Project contains different task and duration
- Employee has to maintain the daywise timesheet for the task completed

Pharmacy Store Management

- Store has many branches, pharmacist as an employee in it
- Drug manufacturing company supplies drugs at different branches
- Patient come with a doctor’s prescription
- Pharmacist sell drugs
- The store manages information relate to the doctor, the patient as a customer, the manufacturer and the pharmacist as an employee – branch wise

E-R Diagram for Pharmacy Store Information



ER-Diagram

Unit-2 Cont….

ER Model to Relational Model

- ER diagrams mainly comprise of – Entity and its attributes Relationship, which is association among entities.

STEP 1:

- Mapping Entity Process *Algorithm*
  - Create table for each entity.
  - Entity's attributes should become fields of tables with their respective data types.
  - Declare primary key.

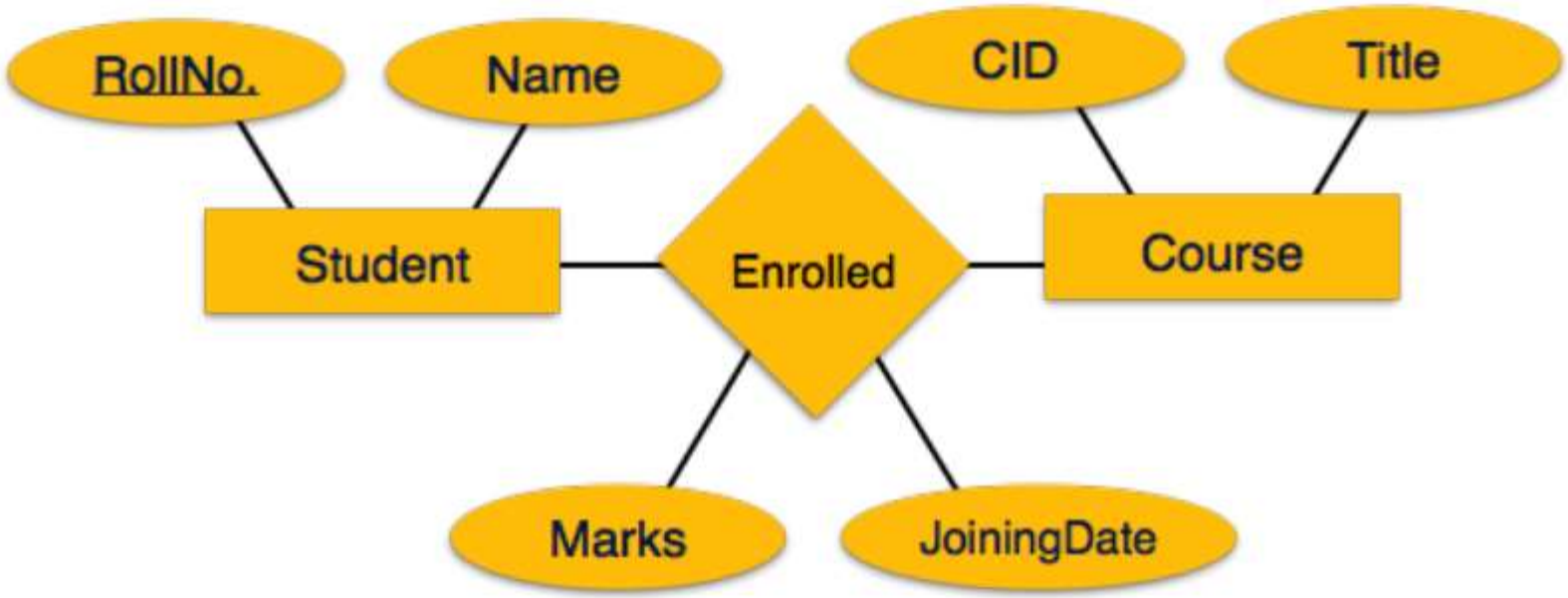
  - Example: We create the relations EMPLOYEE, DEPARTMENT, and PROJECT in the relational schema corresponding to the regular entities in the ER diagram.
    - SSN, DNUMBER, and PNUMBER are the primary keys for the relations EMPLOYEE, DEPARTMENT, and PROJECT as shown.

ER Model to Relational Model (Cont…)

STEP 2:

- Mapping Relationship Process *Algorithm*
  - Create table for a relationship if it is a **n:m (many to many)relationship**.
  - Add the primary keys of all participating Entities as fields of table with their respective data types.
  - If relationship has any attribute, add each attribute as field of table.
  - Declare a primary key composing all the primary keys of participating entities.
  - Declare all foreign key constraints.

  - Example: The M:N relationship type WORKS_ON from the ER diagram is mapped by creating a relation WORKS_ON in the relational database schema.
    - The primary keys of the PROJECT and EMPLOYEE relations are included as foreign keys in WORKS_ON and renamed PNO and ESSN, respectively.
    - Attribute HOURS in WORKS_ON represents the HOURS attribute of the relation type. The primary key of the WORKS_ON relation is the combination of the foreign key attributes {ESSN, PNO}.



Create a table for Enrolled with fields:
(Roll_No, CID, Marks, JoiningDate)

Mapping Relationship type

- **Mapping of Binary 1:1 Relation Types**
- For each binary 1:1 relationship type R in the ER schema, identify the relations S and T that correspond to the entity types participating in R.
- There are three possible approaches:

1. Foreign Key approach: Choose one of the relations-say S-and include a foreign key in S the primary key of T. It is better to choose an entity type with total participation in R in the role of S.
   - Example: 1:1 relation MANAGES is mapped by choosing the participating entity type DEPARTMENT to serve in the role of S, because its participation in the MANAGES relationship type is total.
2. Merged relation option: An alternate mapping of a 1:1 relationship type is possible by merging the two entity types and the relationship into a single relation. This may be appropriate when both participations are total.
3. Cross-reference or relationship relation option: The third alternative is to set up a third relation R for the purpose of cross-referencing the primary keys of the two relations S and T representing the entity types.

Mapping Relationship type

- **Mapping of 1 : N Relationship**
- For each regular binary 1:N relationship type R, identify the relation S that represent the participating entity type at the N-side of the relationship type.
- Include as foreign key in S the primary key of the relation T that represents the other entity type participating in R.
- Include any simple attributes of the 1:N relation type as attributes of S.
  - Example: 1:N relationship types WORKS_FOR, CONTROLS, and SUPERVISION in the figure.
  - For WORKS_FOR we include the primary key DNUMBER of the DEPARTMENT relation as foreign key in the EMPLOYEE relation and call it DNO.

Mapping Relationship type

- **Mapping of N-ary Relationship.**
  - For each n-ary relationship type R, where n>2, create a new relationship S to represent R.
  - Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types.
  - Also include any simple attributes of the n-ary relationship type (or simple components of composite attributes) as attributes of S.

- **Example:** The relationship type SUPPY in the ER on the next slide.
  - This can be mapped to the relation SUPPLY shown in the relational schema, whose primary key is the combination of the three foreign keys {SNAME, PARTNO, PROJNAME}

FIGURE 4.11
Ternary relationship types. (a) The SUPPLY relationship.

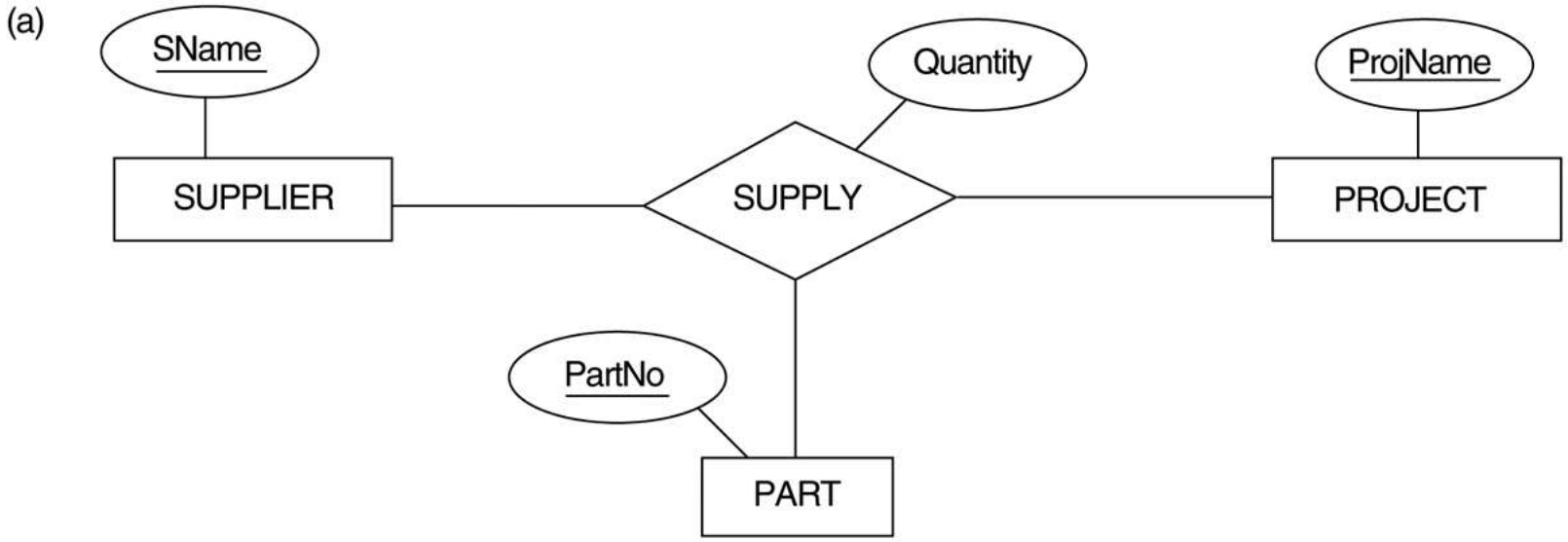


FIGURE 7.3
Mapping the *n*-ary relationship type SUPPLY from Figure 4.11a.

SUPPLIER

SNAME	. . .

PROJECT

PROJNAME	. . .

PART

PARTNO	. . .

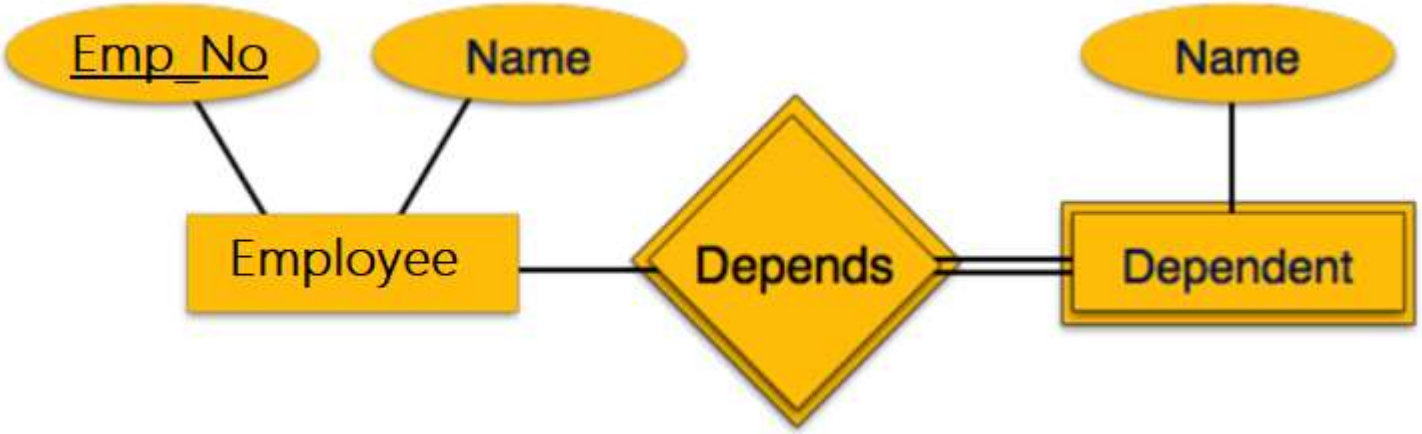
SUPPLY

SNAME	PROJNAME	PARTNO	QUANTITY

ER Model to Relational Model (Cont…)

STEP 3:

- Mapping Weak Entity Sets Process *Algorithm*
  - Create table for weak entity set.
  - Add all its attributes to table as field.
  - Add the primary key of identifying entity set.
  - Declare all foreign key constraints.



ER-to-Relational Mapping Algorithm (contd.)

- **Step 4: Mapping of Multivalued attributes.**
  - For each multivalued attribute A, create a new relation R.
  - This relation R will include an attribute corresponding to A, plus the primary key attribute K-as a foreign key in R-of the relation that represents the entity type of relationship type that has A as an attribute.
  - The primary key of R is the combination of A and K. If the multivalued attribute is composite, we include its simple components.

- **Example:** The relation DEPT_LOCATIONS is created.
  - The attribute DLOCATION represents the multivalued attribute LOCATIONS of DEPARTMENT, while DNUMBER-as foreign key-represents the primary key of the DEPARTMENT relation.
  - The primary key of R is the combination of {DNUMBER, DLOCATION}.

Example

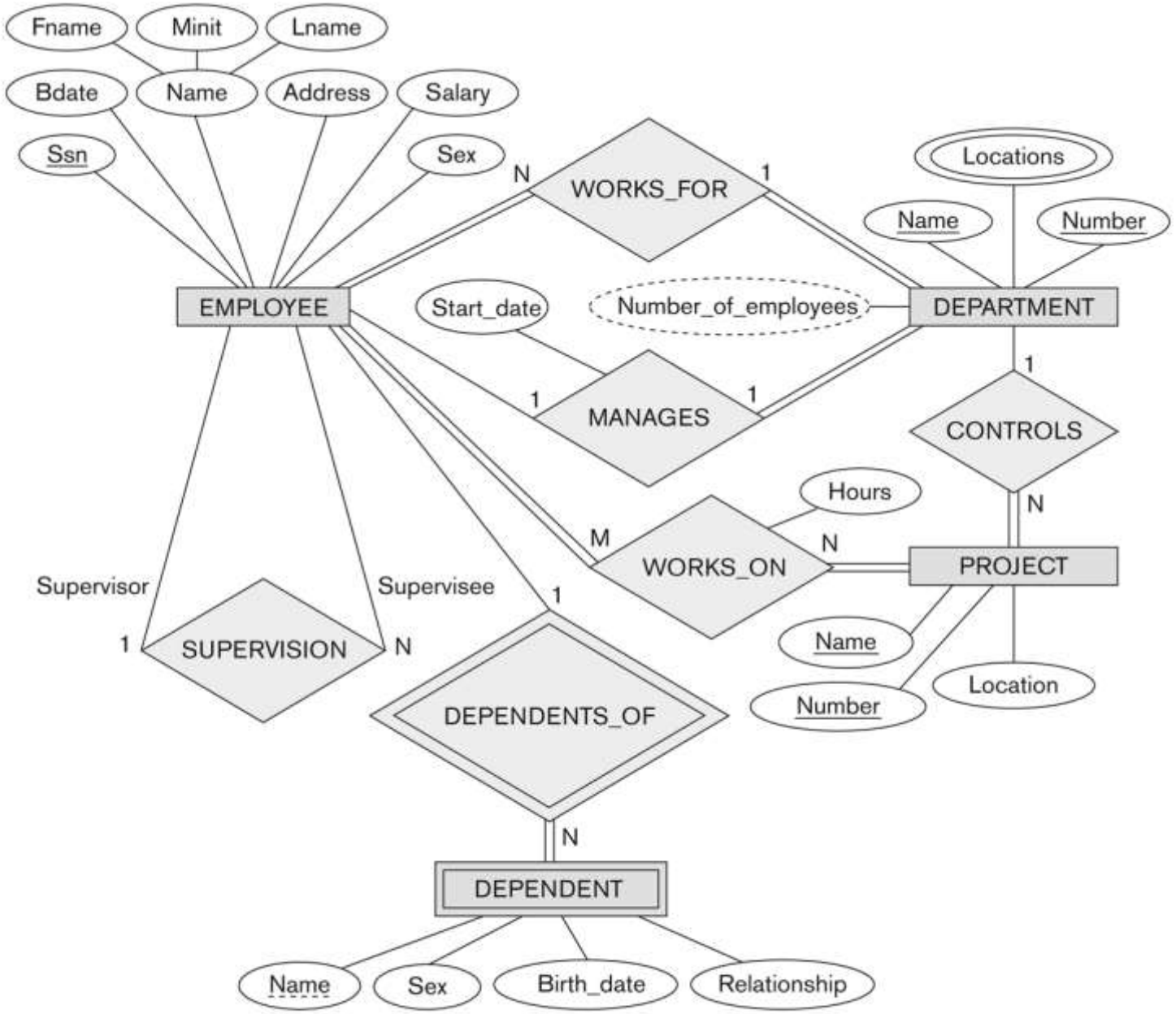


Figure 3.2
An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

FIGURE 7.2
Result of mapping the COMPANY ER schema into a relational schema.

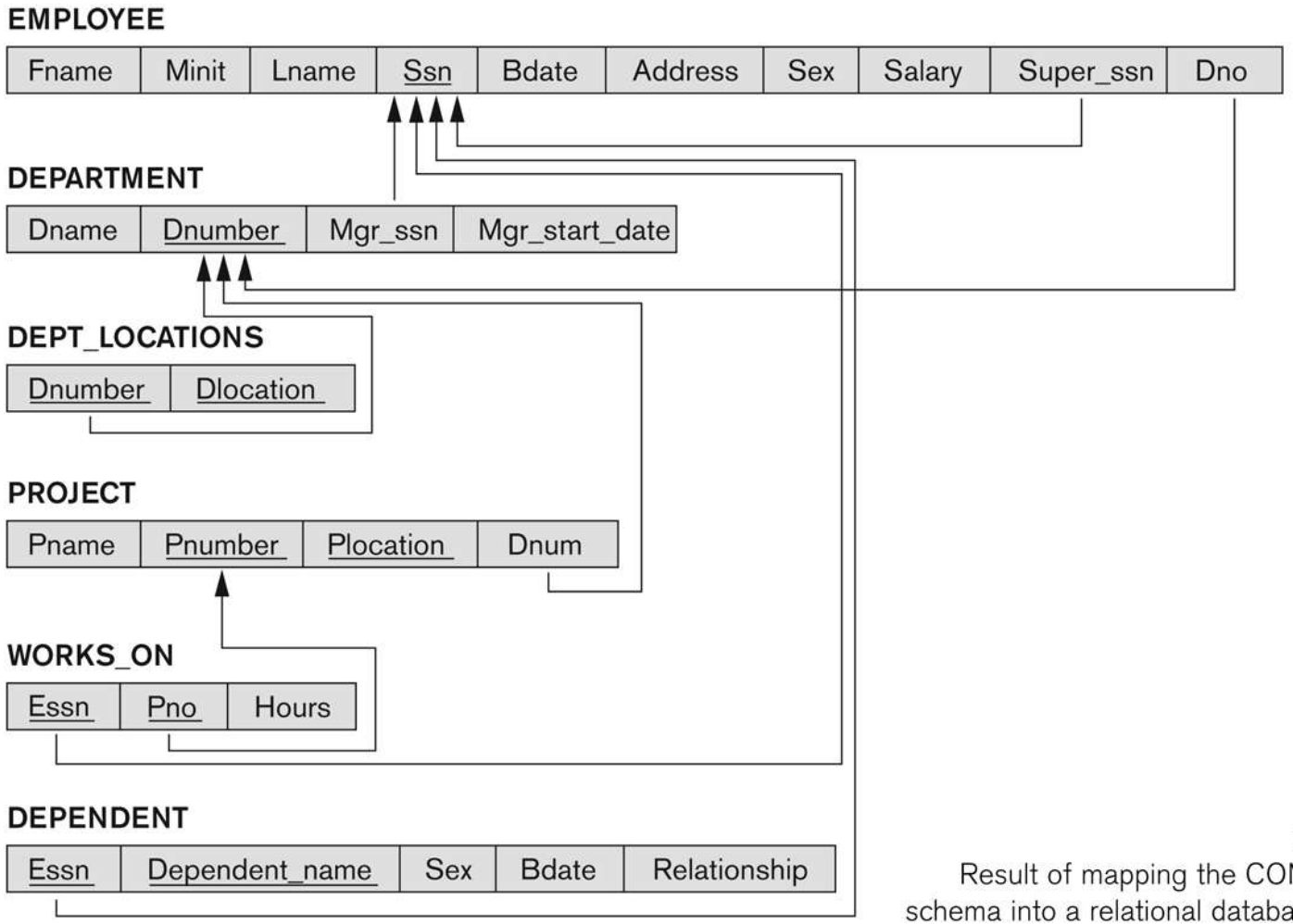


Figure 7.2
Result of mapping the COMPANY ER schema into a relational database schema.

Example

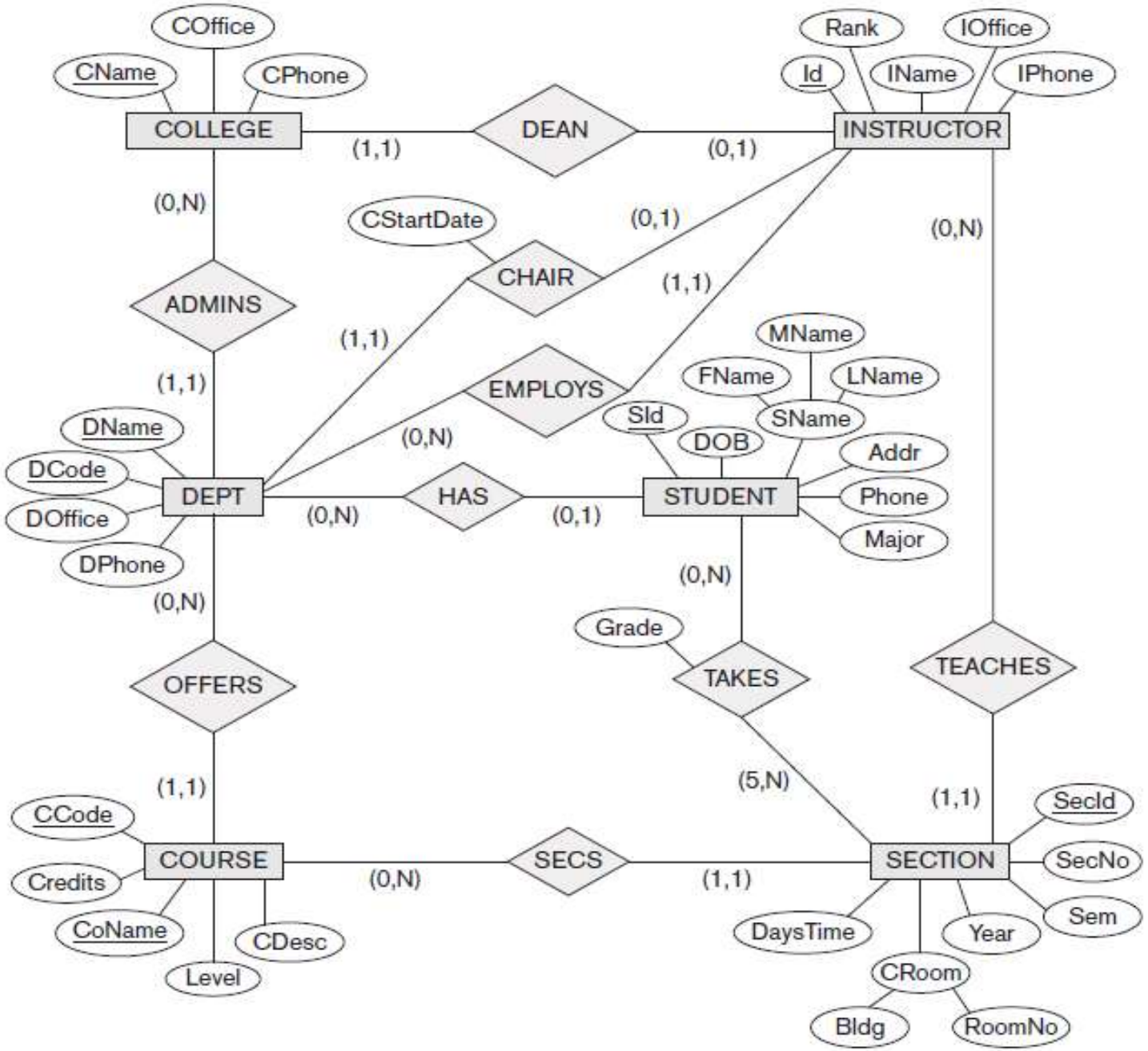


Figure 3.20
An ER diagram for a UNIVERSITY database schema.

Mapping Exercise

Exercise 7.4.

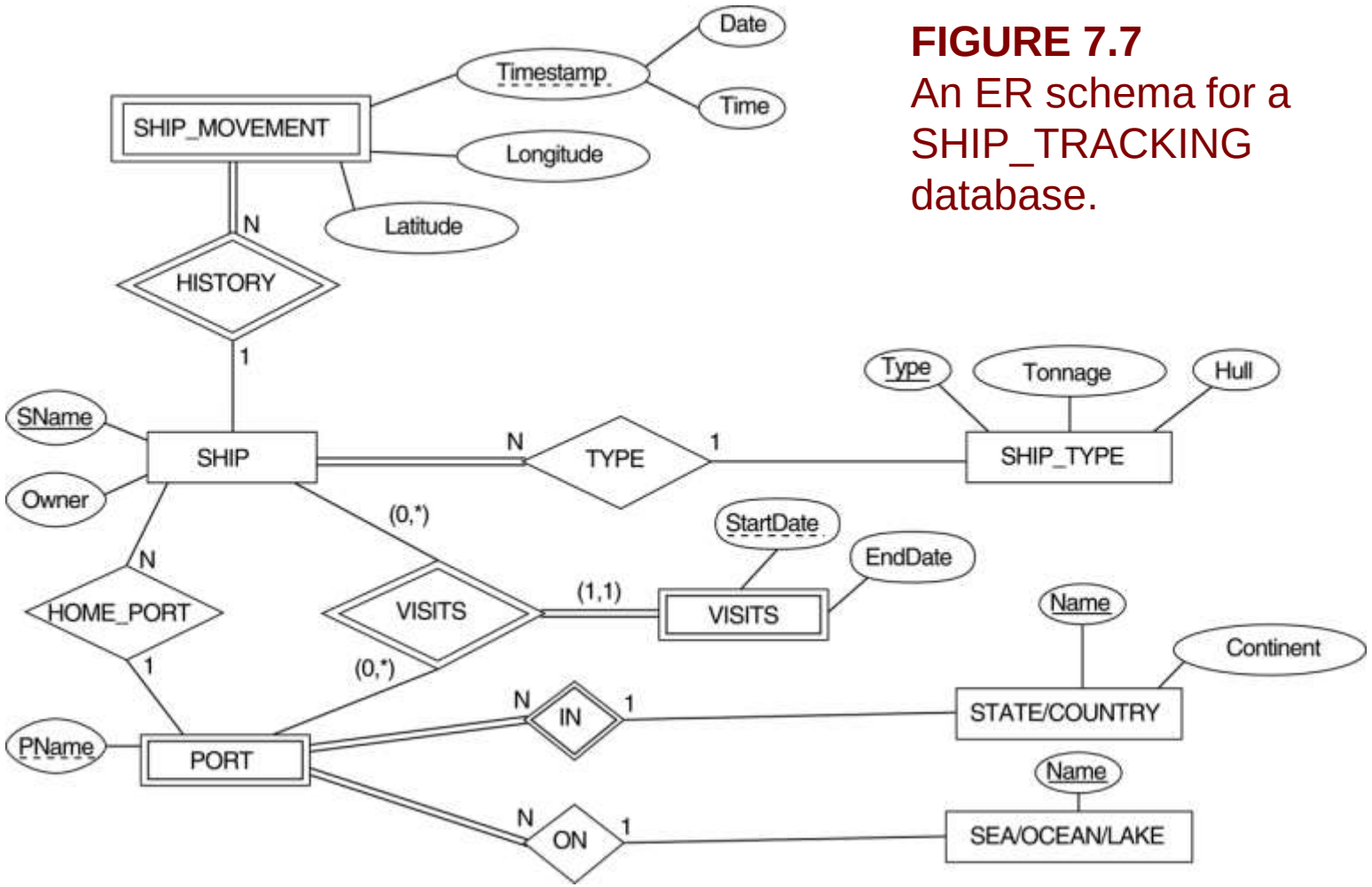


FIGURE 7.7
An ER schema for a SHIP_TRACKING database.

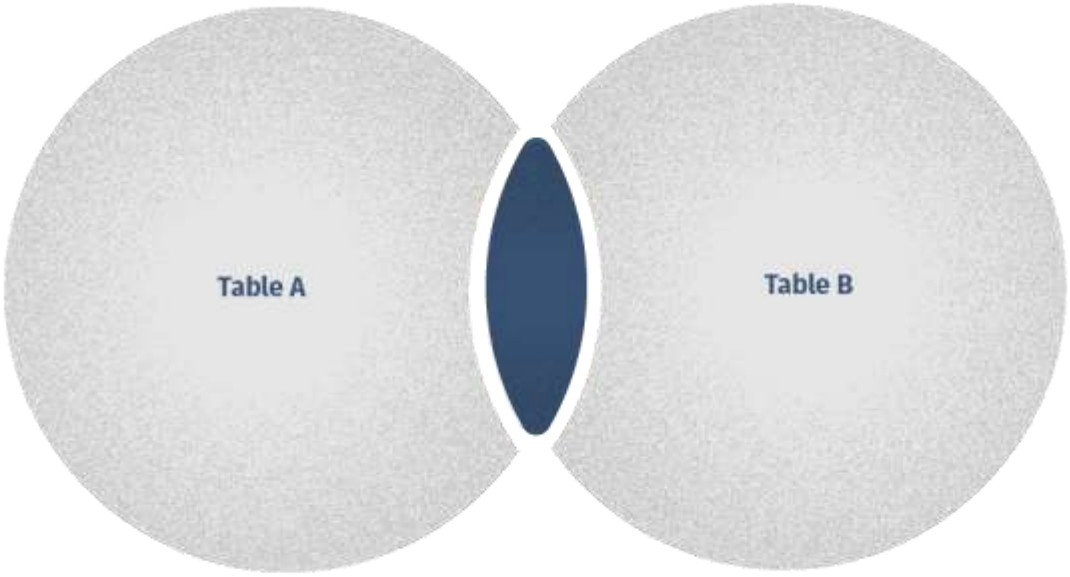
Practical - JOIN

- In relational databases, such as SQL Server, Oracle, MySQL, and others, data is stored in multiple tables that are related to each other with a common key value. Accordingly, there is a constant need to extract records from two or more tables into a results table based on some condition. In SQL Server, this can be easily accomplished with the SQL JOIN clause.

- JOIN is an SQL clause used to query and access data from multiple tables, based on logical relationships between those tables.
- In other words, JOINS indicate how SQL Server should use data from one table to select the rows from another table.

Inner join

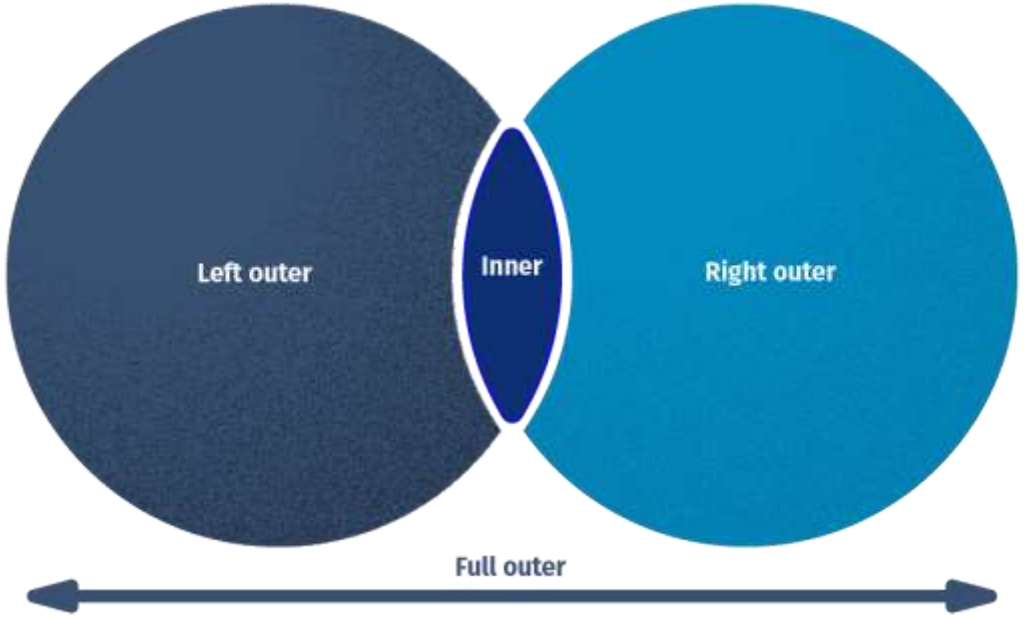
- INNER JOIN statement returns only those records or rows that have matching values and is used to retrieve data that appears in both tables.



Inner Join

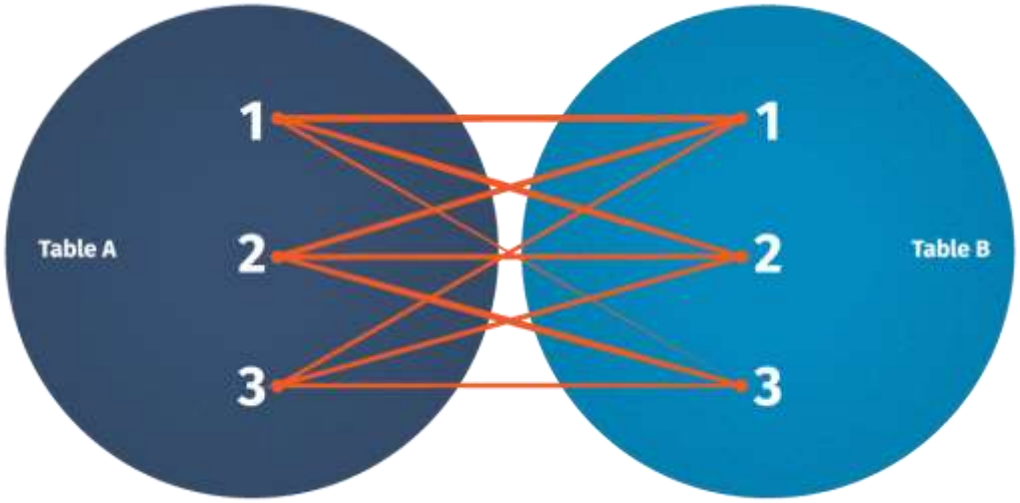
Outer Join – Left Outer and Right Outer

- OUTER JOIN, will retrieve not only the matching rows but also the unmatched rows as well.



Cross Join

- The CROSS JOIN command in SQL, also known as a cartesian join, returns all combinations of rows from each table. Envision that you need to find all combinations of size and color. In that case, a CROSS JOIN will be an asset. Note, that this join does not need any condition to join two tables. In fact, CROSS JOIN joins every row from the first table with every row from the second table and its result comprises all combinations of records in two tables.



Cross Join

Syntax

```
SELECT Table1.Column1,Table1.Column2,Table2.Column1,....
FROM Table1
INNER JOIN Table2
ON Table1.MatchingColumnName = Table2.MatchingColumnName;

SELECT Table1.Column1,Table1.Column2,Table2.Column1,....
FROM Table1
FULL JOIN Table2
ON Table1.MatchingColumnName = Table2.MatchingColumnName;
```

Similarly can write for Left Join an Right Join